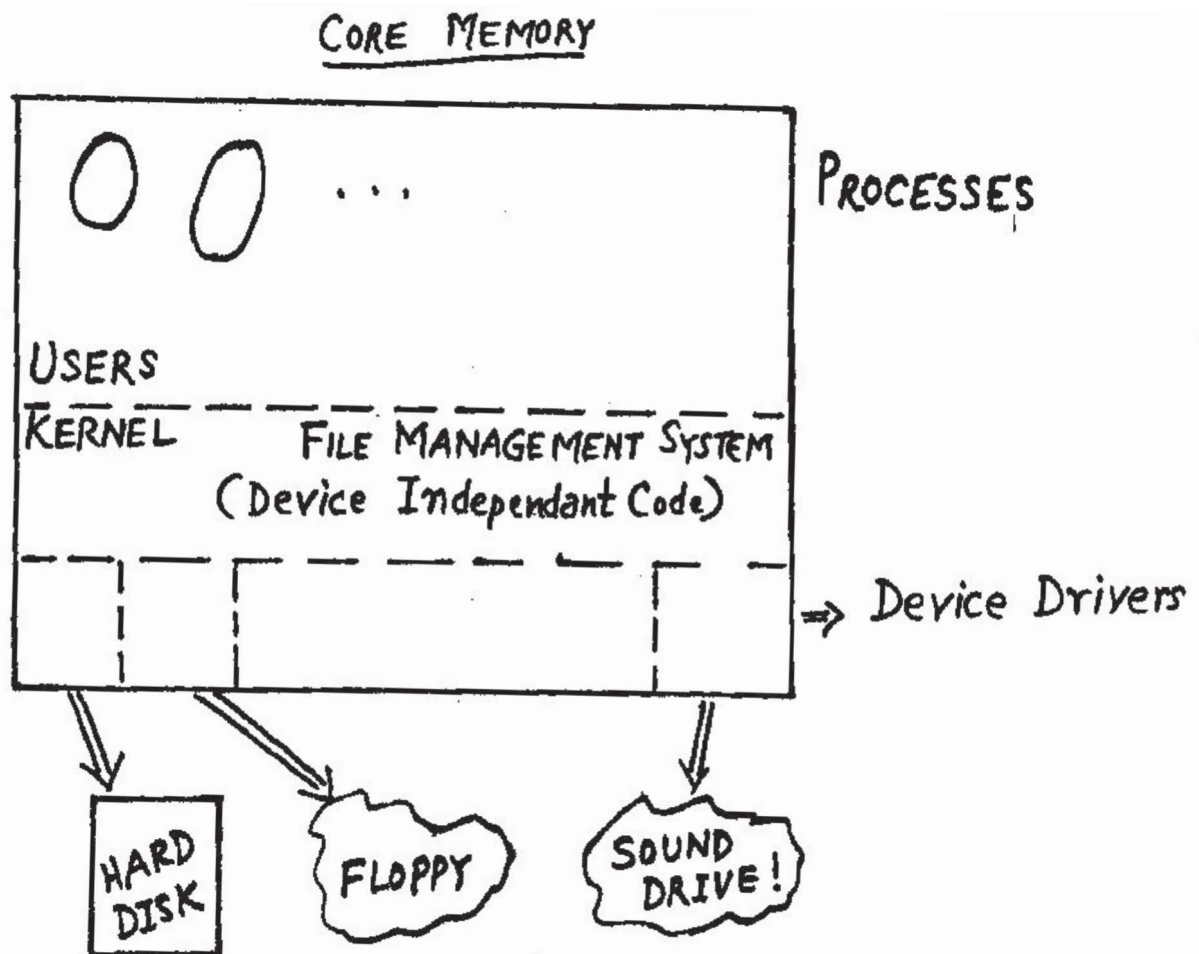




Contains: Cylinders Tracks Sectors

Q: Why?

Logically:  :: Partitions

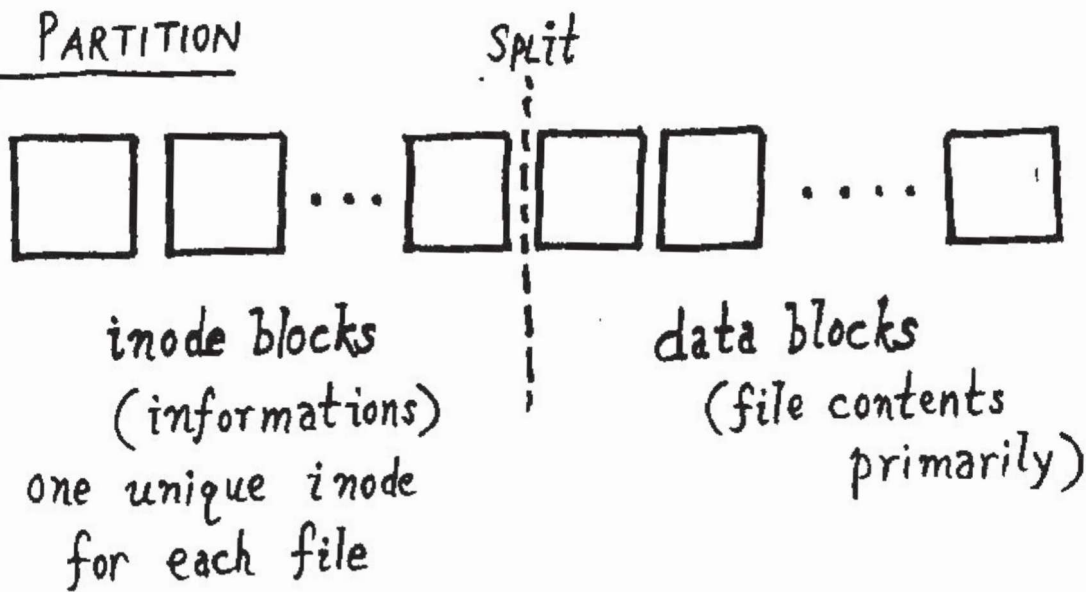
↓ Each Partition

A Partition:  Blocks

Processes want to see:



File system tree
presented by
file management
system.

A PARTITIONINODE INFO.

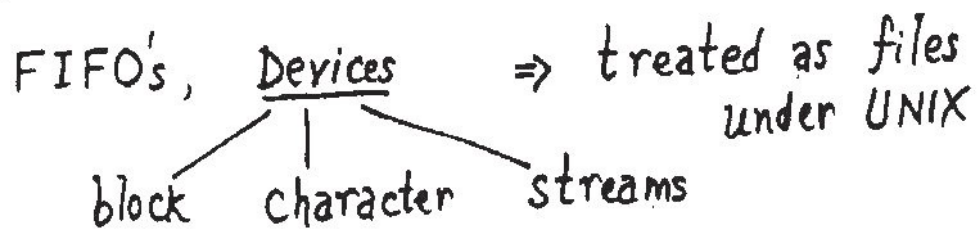
OWNER : uid : user identification number
(an unsigned short)
(Max: 65536 users)

PERMISSIONS: $\underbrace{rwx}_{\text{for owner}} \underbrace{rwx}_{\text{for group to which owner belongs}} \underbrace{rwx}_{\text{for others}}$

FILE SIZE : No File End marker under UNIX

file size : an unsigned long
(Max File Size: 4 GB)

: refers to ordinary files
& directory files

INODE INFOFILE SIZE

A pipe: bytes are coming & going
(pipe size not significant)

Keyboard: no limit to # of chars. to be typed

FILE SIZE: major # : minor #
(class) : (device) $\Rightarrow$ info about a FIFO for FIFO file

identify
device driver

(device #): $\Rightarrow$ for devices:
identify class
& identify device
within class

minor # : 8 bits (256 devices / class)
16 bits (64K devices / class)

BLOCK NUMBERS:

A book : consecutive pages in a chapter

A file : blocks not necessarily consecutive
(because of dynamic growth)

Block Number List : needed (ordinary/
directories)

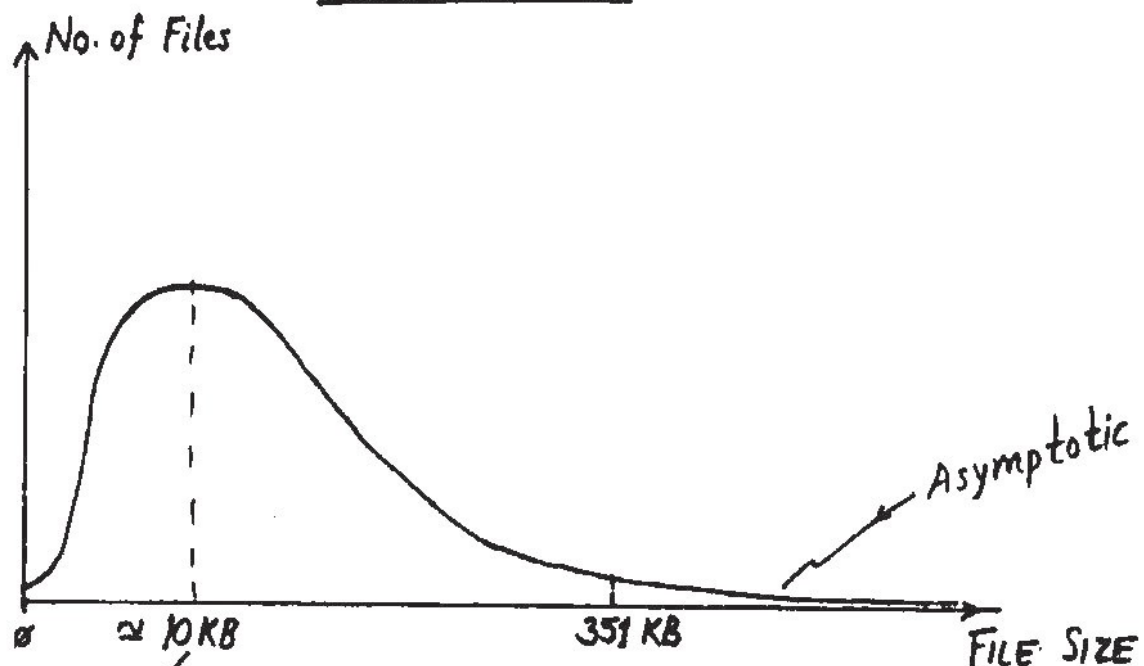
INODE INFOBLOCK NUMBERS

A Sample File : 100 MB on 551K file system
 : 100 K blocks (need not be consecutive)

(Block # : 3 bytes $\Rightarrow$ Max. 16 M blocks/partitions)
 : 300 K bytes needed for block # list
 : 300 blocks for list

Option 1 : Fixed size inode blocks (Wastage)

Option 2 : Variable size inode blocks (Management)

A TREND GRAPH

depends on
 kind of File System
 (An O.O.M. size)

inode containsSIZE

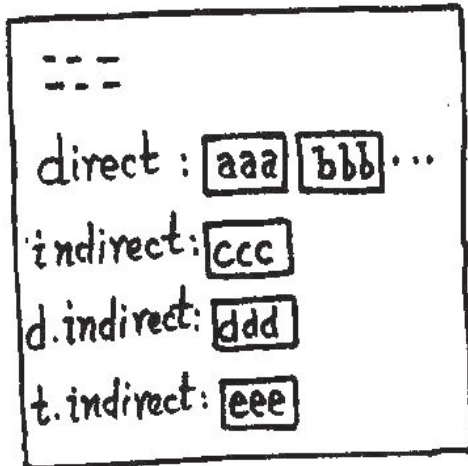
10 : direct data block numbers: 10KB
 1 : indirect data block # : + 341 KB
 1 : double indirect #: + 341 * 341 KB
 1 : triple indirect # :
 + 341 * 341 * 341 KB

INODE INFO

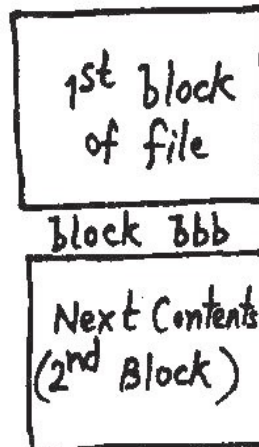
BLOCK NUMBERS

SAMPLE BLOCKS for a file:

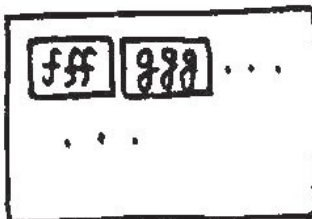
inode block



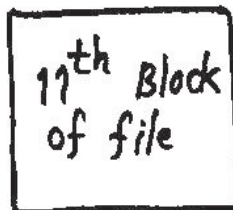
block aaa



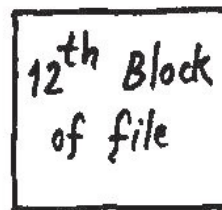
block ccc



block fff

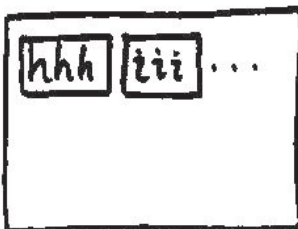


block ggg



... till 351st block

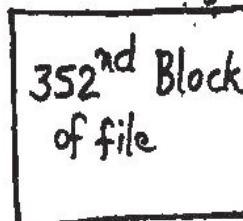
block ddd



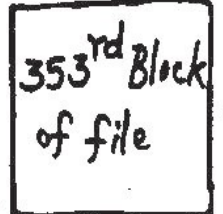
block hhh



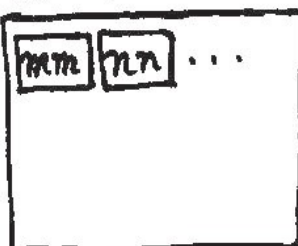
.. block jjj



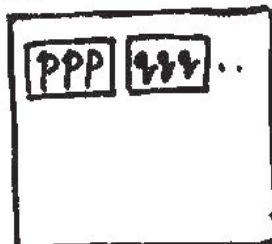
block kkk



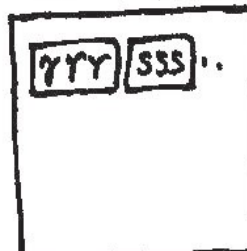
block eee



block mm



block ppp



block rrr



INODE INFOBLOCK NUMBERS

ACCESS TIMES: to read any part of any file
(assume : inode info in memory)

First 10 KB of a file : 1 Disk I/O

Next 341 KB : 2 Disk accesses needed

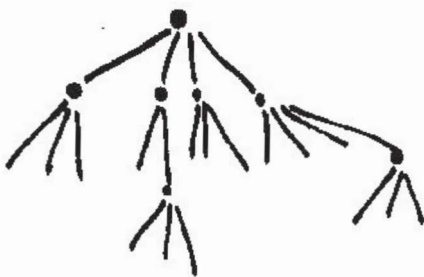
Next ~ 100 MB : 3 Disk accesses

Beyond this : 4 Disk accesses

MOSTLY : 1 / 2 Disk accesses needed

HIERARCHY representation

A partition : Looks like a tree of files



A directory :



Contains: (a) a set of branches
(b) where the branches lead to
(files/directories)

↓
uniquely identified by
inode number
(2 bytes)

A Partition:



inode blocks
(default : 25 %)

for ordinary files, dir, FIFO,
devices

data blocks

file contents
& indirect data blocks

INODE INFOHIERARCHY

Directory Contents: for each branch: A LINK

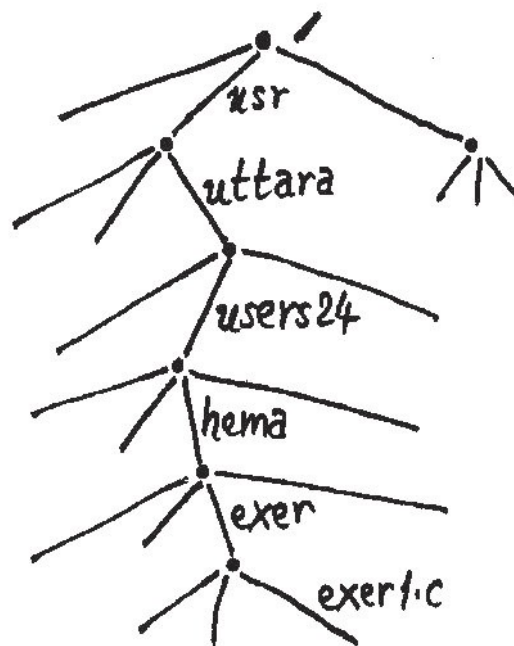
(a) inode number (2 bytes)

(b) branch name (14 bytes)

These are in directory data blocks $\Rightarrow$ Records

Directory Size: multiple of 16.

(Direct data blocks: 10 KB $\Rightarrow$ 640 links)



Root's inode: Fixed
Typically: 1

A SAMPLE CONTENTS

Blk 0



Blk 1



/data blk

xx	usr
xx	bin
xx	lib
1	...

/usr data

xx	lib
xx	bin
xx	uttara

/usr/uttara

etc.

HOME: 'usr/uttara/users24/hema
inode # of this stored as Current Working Dir.

INODE INFOHIERARCHYRELATIVE PATHNAMES

inode no. of Working Directory
stored as part of process information

Ex: `exer/exer1.c` as pathname

↙
one entry in
data block of current
working directory

⇒ gives inode #
& hence data blocks

↘
one entry in
data block of the
directory `exer`

⇒ gives inode #
& hence data blocks

DIRECTORY SIZES

multiple of 16

: minimum : 32 bytes { • entry
 .. entry

Ex: `../chanda/pgm.c`

↙
an entry in the
data block of working directory

⇒ gives inode # of parent directory

: can be used to locate data blocks of
parent directory

Question: Why • entry needed in directories ?

INODE INFOHIERARCHY

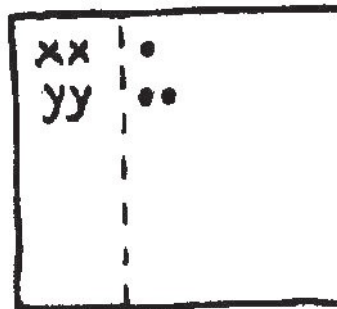
pwd(1) : print working directory
only inode # known $\Rightarrow$ print abs. pathname

Ex: print $\Rightarrow$ /usr/uttara/users24/hema

inode of
Working Dir.

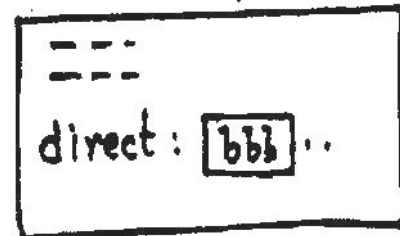


block aaa

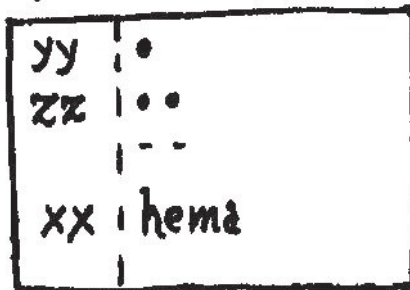


xx: self inode

Block yy: parent inode



Block bbb
(Parent data block)



hema: name against the
self inode number in
parent data block

A File $\Rightarrow$ Like a baby : pathname component
in parent data block.

pwd $\Rightarrow$ Write components backwards.

Parent's name $\Rightarrow$ Ask his parent. (inode # is zz)

/ data block $\Rightarrow$ Adam block : Self is parent.

i.e. • and •• contain same entry (usually 1)

HIERARCHYDIRECTORY ENTRY

Entry is : struct direct in sys/direct.h
One catch: for reading a directory like a file

(NOTE: Directory writing is prohibited;
 Happens when file is created)

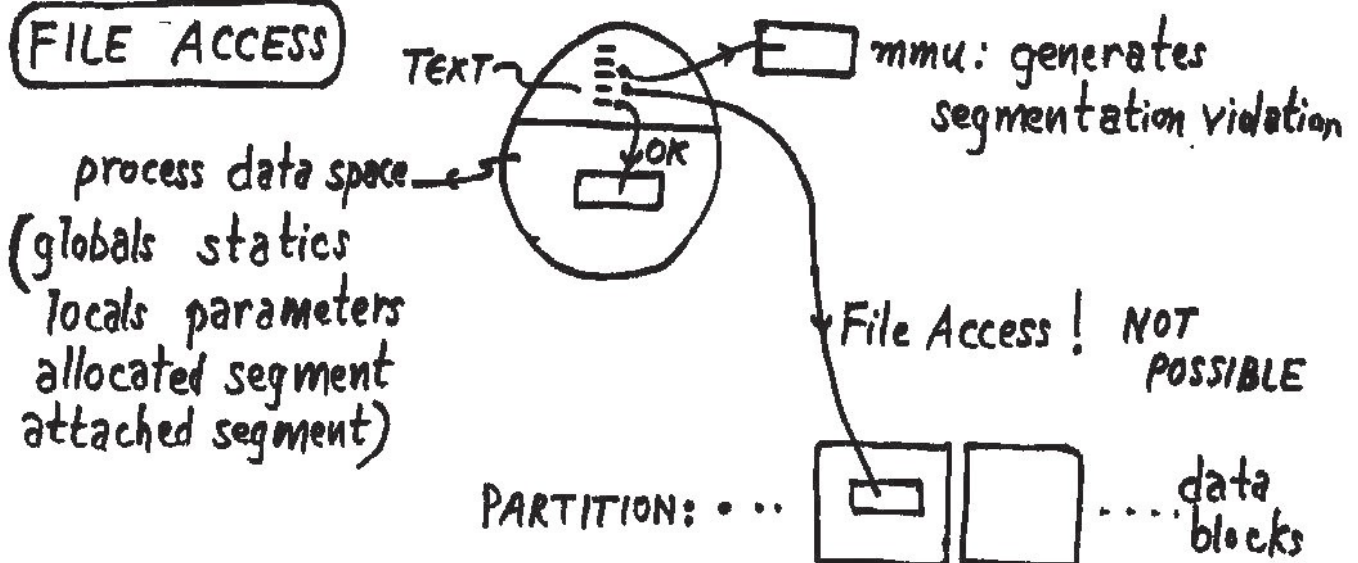
All names < 14 chars $\Rightarrow$ NUL in the end
 names = 14 chars $\Rightarrow$ No NUL @ end

Directory execute permission $\Rightarrow$ Search permission

Current Directory File Access

inode block of current directory in core memory.
 Access inode# of file, read inode block of file
 and then data blocks of file

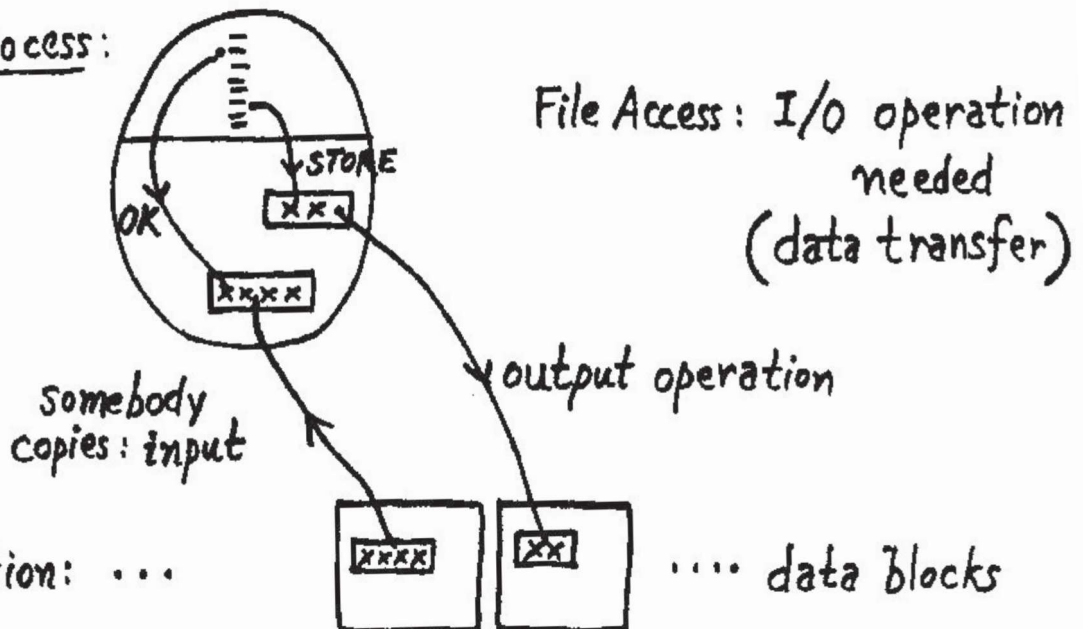
ANALOGY: File System is like a Shop

FILE ACCESS

FILE ACCESS

Mohammad cannot go to the mountain
 $\Rightarrow$ Let mountain come to Mohammad

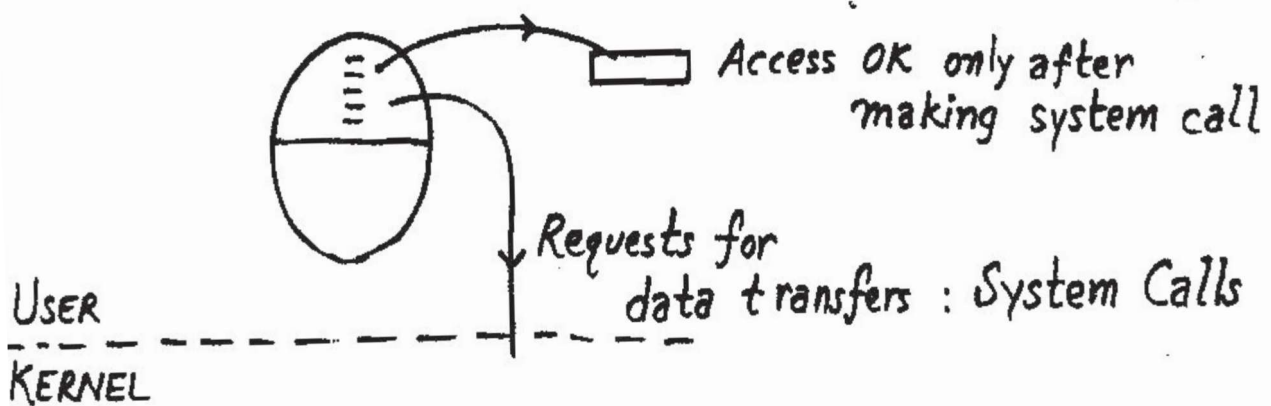
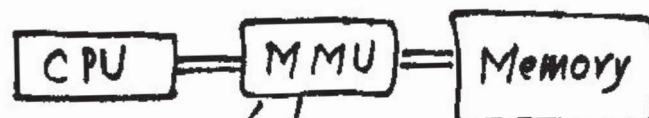
A Process:



Partition: ...

.... data blocks

Data transfers $\Rightarrow$ CANNOT be done by process text
 CAN ONLY be done by kernel text segment

**M.M.U.**

shuts up if CPU executing kernel text
 generates segmentation violation when CPU executes process text

SYSTEM CALLSCPU modes

User mode
(mmu is watching)

kernel mode
(supervisory mode
privileged mode)

Study of system calls $\Rightarrow$ Very important

Question: How many system calls are there?
(different answer for different versions of UNIX)

Use: National Agenda: Common Maximum Programme
(Highest Common Factor)

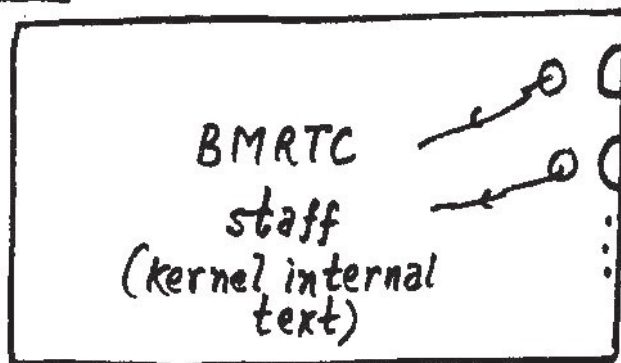
60 - 70 system calls $\Rightarrow$ Most systems programs covered
 $\Rightarrow$ Programs portable

FAMILIES: File Management System Family
: read(2) write(2)

Process Management Family

I/O Management Family

IPC Family etc. $\Rightarrow$ Aid to study

ANALOGY

Customers: Make Requests
 $\Downarrow$
like processes

(Counters): System Call Families

SYSTEM CALLS interface from process text
to rest of the kernel
i.e. just entries into kernel text segment

Studying system calls $\Rightarrow$ Sufficient to understand
internals of UNIX
(PMRTC can be set up)

Analogy: Sense organs of human body

FMS Implementation: System calls available
need to be studied

bottomline: superfast access times

NOTE: may not be possible always

Principle:

What you need to do frequently $\Rightarrow$ do very fast

What you need infrequently $\Rightarrow$ No need for speed

(Total performance not affected)

NOTE: FMS is precompiled part of kernel.

Hence any eventual kind of activity
must be supported.

HAVE: high speed for activities that most processes
want to do on most files most of the time.

FMS Issues

Security: must be absolute
but must be non-obtrusive

obtrusive security: Breathing down the neck.
We cannot have security check for each
data transfer of a file.
One single check done in the beginning.

Multiple Accesses

Question: Can 2 processes read from same file ?

Question: Can multiple processes write into
same file ?

Ex: attendance register ; logbook

Question: Same process accessing multiple files
concurrently ?

Question: How many files can a process access ?
Like reading books in a library

Question: Same process accessing same file
in multiple places ?
Like: Trunk booking